CHAPTER 1

# Pre-Training, Scaling Laws, and the Evolution of Large Language Models

Chapter **??** analyzed the structural taxonomy of Transformer variants, detailing how encoder-only, encoder-decoder, and decoder-only topologies specialize for distinct linguistic representations and sequence generation tasks. While architectural topology dictates how tokens attend to one another within the multi-head attention mechanism, the emergent cognitive, semantic, and reasoning capabilities of modern foundation models are governed by the scale of self-supervised pre-training, parameter capacity, and compute allocation.

The evolution of modern large language models began with the empirical demonstration that unsupervised autoregressive pre-training over diverse text corpora provides a universal task-agnostic initialization that transfers to downstream tasks. Subsequent scaling experiments revealed that language model loss follows precise mathematical power laws across parameter count, token volume, and computational budget, triggering an industry-wide race toward massive parameter regimes. However, subsequent empirical evaluations demonstrated that simply increasing parameter counts without proportionally scaling training tokens produces severely undertrained models, establishing the compute-optimal scaling frontier and laying the foundation for modern open foundation models.

This chapter provides an exhaustive theoretical and empirical analysis of the evolution of large language models, their pre-training methodologies, and the scaling laws governing their performance. We begin in Section 1.1 by formalizing the Generative Pre-trained Transformer (GPT-1), dissecting its two-stage pre-training and supervised fine-tuning framework with delimiter tokens. Section 1.2 examines GPT-2, its zero-shot multi-task paradigm, WebText corpus curation, benchmark evaluations, and the breakthrough empirical scaling laws of Kaplan et al. Section 1.3 deconstructs GPT-3, contrasting supervised fine-tuning with in-context learning across zero-shot, one-shot, and few-shot regimes, and detailing its empirical performance against task-specific state-of-the-art baselines. Section 1.4 analyzes the race to massive scale, the undertraining diagnosis, and the three empirical approaches of the DeepMind Chinchilla framework that formalized compute-optimal pre-training. Finally, Section 1.5 examines the modern paradigm shift toward token saturation, inference economics, and the rise of open foundation model families.

## 1.1 GPT-1: Generative Pre-Training and Supervised Adaptation

Prior to 2018, natural language processing was characterized by fragmented, task-specific architectures. Researchers predominantly trained recurrent neural networks (LSTMs, GRUs) or convolutional networks on supervised datasets from scratch, relying on static word embeddings such as Word2Vec or GloVe for initialization. While semi-supervised approaches such as ELMo contextualized word representations, the underlying task-specific model architectures still required bespoke engineering for each target task.

Radford et al. [**?** ] resolved this architectural fragmentation by introducing the Generative Pre-trained Transformer (GPT-1). GPT-1 demonstrated that a single, universal task-agnostic Transformer architecture could be pre-trained using self-supervised causal language modeling on unannotated text, followed by minimal supervised fine-tuning on labeled downstream benchmarks.

### 1.1.1 Architectural topology and pre-training setup

The core architecture of GPT-1 adapts the canonical Transformer decoder proposed by Vaswani et al. [**?** ], excising the encoder-decoder cross-attention sub-layers because no external source sequence is encoded. The model consists of a 12-layer decoder-only stack with the following dimensional specifications:

- **Layer and Head Configuration**: $L = 12$ Transformer decoder blocks, each containing $H = 12$ self-attention heads with per-head projection dimension $d_{\text{head}} = d_{\text{model}}/H = 64$.
- **Hidden Dimensionality**: Hidden representation dimension $d_{\text{model}} = 768$, with position-wise feed-forward intermediate dimensionality $d_{\text{ff}} = 3072$ (a 4× expansion ratio).
- **Parameter Scale**: Total parameter count of $N \approx 117$M parameters.
- **Tokenization and Embeddings**: Byte-Pair Encoding (BPE) subword tokenization with a vocabulary of 40,000 merge operations. Rather than fixed sinusoidal encodings, GPT-1 employs learned absolute positional embeddings up to a maximum context window of $L = 512$ tokens.

For its pre-training corpus, GPT-1 utilized the BooksCorpus dataset. Comprising approximately 7,000 unique unpublished books (amounting to ≈ 5 GB of text) across diverse literary genres (adventure, science fiction, romance), BooksCorpus provided continuous spans of long-form, coherent prose. This allowed the self-attention mechanism to learn long-range semantic and syntactic dependencies across contiguous discourse, rather than sentence-level fragments.

In the first stage of the GPT-1 framework, the network is trained on an unannotated corpus of tokens $\mathcal{U} = (u_1, u_2, \ldots, u_N)$ via standard autoregressive language modeling. Given a conditioning context window of size $k$, the network maximizes the causal log-likelihood:

$$\mathcal{L}_1(\mathcal{U}) = \sum_{i=1}^{N} \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}; \boldsymbol{\Theta}), \tag{1.1}$$

where $\boldsymbol{\Theta}$ denotes the complete parameter tensor of the Transformer decoder.

Figure 1.1 illustrates the forward computation. Formally, given an input token sequence $(u_{i-k}, \ldots, u_{i-1})$, the zeroth representation layer $\mathbf{h}_0$ is constructed by summing token embedding matrix $\mathbf{W}_e \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ and positional embedding matrix $\mathbf{W}_p \in \mathbb{R}^{L \times d_{\text{model}}}$:

$$\mathbf{h}_0 = \mathbf{U}\mathbf{W}_e + \mathbf{W}_p, \tag{1.2}$$

where $\mathbf{U} \in \{0, 1\}^{k \times |\mathcal{V}|}$ is the one-hot context matrix.

The representation is then propagated through $L$ identical Transformer decoder layers:

$$\mathbf{h}_l = \text{TransformerBlock}(\mathbf{h}_{l-1}), \quad \forall l \in \{1, \ldots, L\}. \tag{1.3}$$

The conditional probability distribution over vocabulary $\mathcal{V}$ at step $i$ is obtained by projecting the final hidden state $\mathbf{h}_L$ through an explicit unembedding matrix $\mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$:

$$P(u_i \mid u_{i-k}, \ldots, u_{i-1}) = \text{Softmax}(\mathbf{W}_U \mathbf{h}_{L,i-1} + \mathbf{b}_U). \tag{1.4}$$

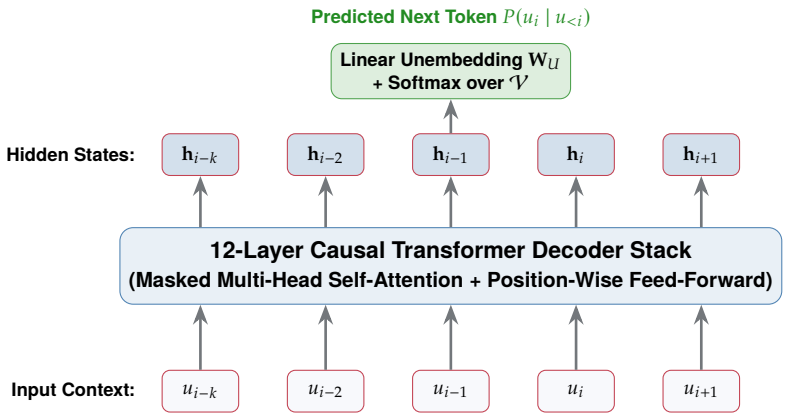



Figure 1.1: The GPT-1 unsupervised pre-training stage [**?** ]. A causal Transformer decoder processes context tokens under a lower-triangular attention mask, projecting each hidden state $\mathbf{h}_{i-1}$ through an unembedding matrix $\mathbf{W}_U$ to maximize the conditional likelihood of the ground-truth next token $u_i$.

### 1.1.2 Supervised task adaptation and delimiter token linearization

Once pre-training is complete, the parameters $\boldsymbol{\Theta}$ provide a general-purpose semantic initialization. In the supervised fine-tuning stage, the model is adapted to a labeled dataset $C$, where each instance consists of an input sequence of tokens $(x_1, \ldots, x_m)$ and a ground-truth target label $y$.

The input sequence is passed through the pre-trained Transformer decoder to obtain the final activation vector of the terminal token, $\mathbf{h}_{L,m} \in \mathbb{R}^{d_{\text{model}}}$. This state is fed into a lightweight linear classification layer parameterized by $\mathbf{W}_y \in \mathbb{R}^{K \times d_{\text{model}}}$:

$$P(y \mid x_1, \ldots, x_m) = \text{Softmax}(\mathbf{W}_y \mathbf{h}_{L,m} + \mathbf{b}_y). \tag{1.5}$$

The supervised objective minimizes the empirical cross-entropy loss:

$$\mathcal{L}_2(C) = -\sum_{(\mathbf{x},y)\in C} \log P(y \mid x_1, \ldots, x_m). \tag{1.6}$$

Critically, Radford et al. observed that incorporating an auxiliary language modeling objective during fine-tuning accelerates convergence, prevents catastrophic forgetting, and improves downstream generalization:

$$\mathcal{L}_3(C) = \mathcal{L}_2(C) + \lambda \mathcal{L}_1(C), \tag{1.7}$$

where $\lambda > 0$ balances supervised classification and self-supervised causal modeling (typically set to $\lambda = 0.5$).

A defining innovation of GPT-1 was the complete avoidance of task-customized neural architectures. As illustrated in Figure 1.2, structured inputs are flattened into linear token sequences bounded by special delimiter tokens:

1. **Text Classification**: The input sequence is bounded by start and extract tokens: $(\texttt{Start}, x_1, \ldots, x_m, \texttt{Extract})$. The linear head reads the final contextual representation corresponding to `Extract`.

2. **Natural Language Inference (NLI)**: Premise and hypothesis sequences are concatenated with a middle delimiter: (`Start`, Premise, `Delim`, Hypothesis, `Extract`).

3. **Semantic Similarity**: Because similarity is symmetric, the pair is processed in both forward and reversed order: (`Start`, $\text{Text}_1$, `Delim`, $\text{Text}_2$, `Extract`) and (`Start`, $\text{Text}_2$, `Delim`, $\text{Text}_1$, `Extract`). The two resulting extract vectors $\mathbf{h}^{(1)}_{L,\text{ext}}$ and $\mathbf{h}^{(2)}_{L,\text{ext}}$ are summed element-wise before classification, ensuring permutation invariance.

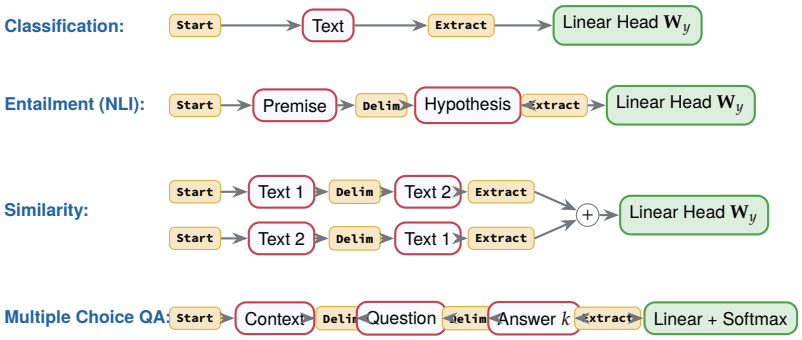



Figure 1.2: Downstream task input transformations using delimiter tokens in GPT-1 [**?** ]. Rather than altering the underlying model architecture, structured inputs are flattened into linear token sequences bounded by `Start`, `Delim`, and `Extract` tokens.

4. **Question Answering and Commonsense Reasoning**: For a context document $z$, question $q$, and candidate answer set $\{a_k\}$, the model forms $K$ independent sequences: $(\texttt{Start}, z, \texttt{Delim}, q, \texttt{Delim}, a_k, \texttt{Extract})$. Each sequence produces an extract logit, and a softmax over all $K$ candidates yields the choice probability.

## 1.2 GPT-2, WebText, and the Empirical Foundation of Scaling Laws

Following the success of GPT-1, Radford et al. [**?** ] released GPT-2. The primary conceptual breakthrough of GPT-2 was the demonstration of *unsupervised multi-task learning*: when trained on a sufficiently diverse and massive corpus, a language model develops zero-shot capabilities across distinct tasks without requiring supervised fine-tuning or parameter updates.

### 1.2.1 Architectural stabilization, model scaling, and WebText curation

While preserving the core causal decoder topology of GPT-1, GPT-2 introduced critical structural modifications to stabilize optimization across deeper layer hierarchies:

- **Pre-Layer Normalization (Pre-LN)**: In GPT-1, Layer Normalization was applied after the residual addition (*Post-LN*): $\mathbf{x}_{l+1} = \text{LN}(\mathbf{x}_l + \text{SubBlock}(\mathbf{x}_l))$. In GPT-2, normalization was shifted to the input pathway of each sub-block (*Pre-LN*):

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \text{SubBlock}(\text{LN}(\mathbf{x}_l)). \tag{1.8}$$

  An additional Layer Normalization layer was added following the final self-attention block prior to the unembedding projection. Pre-LN establishes an unhindered identity highway through the residual stream, preventing gradient vanishing or explosion in deep architectures.

- **Residual Projection Scaling**: To prevent variance accumulation across stacked residual connections at initialization, the weights of residual projection layers ($\mathbf{W}^O$ in self-attention and $\mathbf{W}_2$ in feed-forward networks) were scaled down by a depth-dependent factor:

$$\mathbf{W} \sim \mathcal{N}\left(0, \frac{\sigma^2}{\sqrt{2N}}\right), \tag{1.9}$$

  where $N$ is the number of Transformer layers.

- **Context and Vocabulary Expansion**: The context window was doubled from 512 to $L = 1024$ tokens. To handle arbitrary Unicode sequences without out-of-vocabulary

([UNK]) tokens, GPT-2 introduced Byte-Level Byte-Pair Encoding (BBPE) with a vocabulary of $|\mathcal{V}| = 50,257$.

GPT-2 was released as a family of four model sizes, scaling from 117M to 1.5B parameters:

1. **GPT-2 Small**: $L = 12$ layers, $H = 12$ heads, $d_{\text{model}} = 768$ (117M parameters, identical to GPT-1).
2. **GPT-2 Medium**: $L = 24$ layers, $H = 16$ heads, $d_{\text{model}} = 1024$ (345M parameters).
3. **GPT-2 Large**: $L = 36$ layers, $H = 20$ heads, $d_{\text{model}} = 1280$ (762M parameters).
4. **GPT-2 Extra Large**: $L = 48$ layers, $H = 25$ heads, $d_{\text{model}} = 1600$ (1.5B parameters).

A critical factor in GPT-2's emergent capabilities was the curation of the *WebText* corpus. While common web-scraping repositories (such as Common Crawl) contained hundreds of terabytes of data, raw web scrapes were plagued by automated spam, duplicate content, and incoherent text. To ensure high quality without manual labeling, Radford et al. designed a novel heuristic curation strategy:

1. **Reddit Link Scraping**: Outbound links shared on the social news platform Reddit were extracted.
2. **Human Karma Filtering**: Only links from Reddit posts receiving at least 3 karma (upvotes) were retained. This used human curation as an explicit heuristic filter: the Reddit community effectively pre-screened outbound web content for coherence, informative value, and linguistic quality.
3. **Text Extraction and Deduplication**: From over 45 million links, Wikipedia articles were removed (to avoid contamination of evaluation benchmarks), HTML was stripped, and text was deduplicated.

The resulting WebText dataset yielded approximately 40 GB of cleaned text—an 8× expansion in text volume over BooksCorpus.

### 1.2.2 Emergent zero-shot multi-task generalization and staged release

The central empirical result of GPT-2 was that language models, without any fine-tuning or parameter updates, can execute diverse natural language processing tasks zero-shot by framing the objective as conditional natural language generation:

$$P(\text{Target} \mid \text{Context, Task Description}). \tag{1.10}$$

Rather than training a task-specific classification head, the task instructions and input context are concatenated into a prompt sequence, and the model generates the target completion through standard autoregressive decoding.

To rigorously benchmark this zero-shot transfer capability, Radford et al. evaluated the four GPT-2 model variants across a broad battery of language modeling, cloze completion, reading comprehension, and translation benchmarks without fine-tuning. Table 1.1 presents the comprehensive empirical results reported in Table 3 of Radford et al. [**?** ].

The empirical findings from Table 1.1 established several profound takeaways:

1. **Long-Range Context Sensitivity on LAMBADA**: The LAMBADA dataset tests the ability of language models to resolve long-range discourse dependencies by requiring the model to predict the final word of a passage that cannot be predicted from local context alone. The prior state-of-the-art model achieved a perplexity of 99.8 and an accuracy of 59.23%. GPT-2 1.5B reduced the perplexity by more than an order of magnitude to 8.63, while increasing accuracy to 63.24% zero-shot, demonstrating that scaling parameter count enables the self-attention mechanism to track subtle discourse threads across full narrative paragraphs.

Table 1.1: Zero-shot performance of GPT-2 across model scales compared to prior task-specific state-of-the-art models [**?** ]. No training or supervised fine-tuning was performed for any GPT-2 results.

| Benchmark / Dataset | Prior SOTA | 117M (Small) | 345M (Med) | 762M (Large) | 1.5B (XL) |
|---|---|---|---|---|---|
| **LAMBADA** (Perplexity ↓) | 99.80 | 35.13 | 15.60 | 10.87 | **8.63** |
| **LAMBADA** (Accuracy % ↑) | 59.23 | 45.99 | 55.48 | 60.12 | **63.24** |
| **CBT-CN** (Accuracy % ↑) | 85.70 | 87.65 | 92.35 | **93.45** | 93.30 |
| **CBT-NE** (Accuracy % ↑) | 82.30 | 83.40 | 87.10 | 88.00 | **89.05** |
| **WikiText-2** (Perplexity ↓) | 39.14 | 29.41 | 22.76 | 19.93 | **18.34** |
| **Penn Treebank** (Perplexity ↓) | 46.54 | 65.85 | 47.33 | 40.31 | **35.76** |
| **enwik8** (Bits per byte ↓) | 0.99 | 1.16 | 1.01 | 0.97 | **0.93** |
| **text8** (Bits per character ↓) | 1.08 | 1.17 | 1.06 | 1.02 | **0.98** |
| **WikiText-103** (Perplexity ↓) | 18.30 | 37.50 | 26.37 | 22.05 | **17.48** |
| **1BW (One Billion Words)** (PPL ↓) | 21.80 | 75.20 | 55.72 | 44.57 | **42.16** |
| **CoQA** (Development F1 ↑) | 45.0–54.0* | 43.10 | 49.60 | 52.80 | **55.00** |

*Prior supervised baseline systems on CoQA without pre-training (e.g., DrQA, BiDAF) achieved between 45 and 54 F1 on the dev set, whereas supervised BERT reached 89 F1.

2. **Cloze Performance on Children's Book Test (CBT)**: CBT evaluates language models on predicting missing common nouns (CBT-CN) and named entities (CBT-NE). GPT-2 1.5B surpassed prior supervised models on both benchmarks, achieving 93.30% on CBT-CN (vs. prior SOTA 85.70%) and 89.05% on CBT-NE (vs. prior SOTA 82.30%), without being exposed to task-specific training instances.

3. **Log-Linear Performance Scaling**: Across language modeling benchmarks (WikiText-2, PTB, enwik8, text8), perplexity dropped in a continuous, log-linear relationship as model capacity expanded from 117M to 1.5B parameters. On WikiText-2, GPT-2 1.5B halved the prior state-of-the-art perplexity (from 39.14 down to 18.34).

4. **Zero-Shot Conversational QA and Prompting Emergence**: On the Conversational Question Answering (CoQA) dataset, GPT-2 was evaluated by reading context passages and answering multi-turn dialog questions zero-shot. By formatting the dialog history and appending the terminal prompt `"A:"`, GPT-2 1.5B achieved 55.0 F1 on the development set, matching or outperforming three out of four prior supervised baselines without using a single one of CoQA's 127,000+ training examples. Furthermore, appending `"TL;DR:"` induced abstractive document summarization, while bilingual phrase pairs prompted machine translation on WMT-14 French-English (achieving 11.5 BLEU zero-shot).

Despite these technical successes, the release of GPT-2 was overshadowed by significant governance controversy. Citing concerns regarding the potential dual-use risks of 1.5B-scale generative models—specifically automated generation of deceptive news articles, personalized phishing campaigns, and industrial-scale social media manipulation—OpenAI initially withheld the 1.5B parameter weights. Instead, OpenAI implemented a phased, staged release: the 117M model was published in February 2019, the 345M model in May, the 762M model in August, and finally the complete 1.5B model in November 2019 after partnership studies observed minimal evidence of malicious misuse. This decision sparked intense debate within the machine learning community: critics viewed it as an orchestrated marketing strategy that compromised open scientific reproducibility, while proponents regarded it as a landmark precedent for responsible disclosure and AI governance.

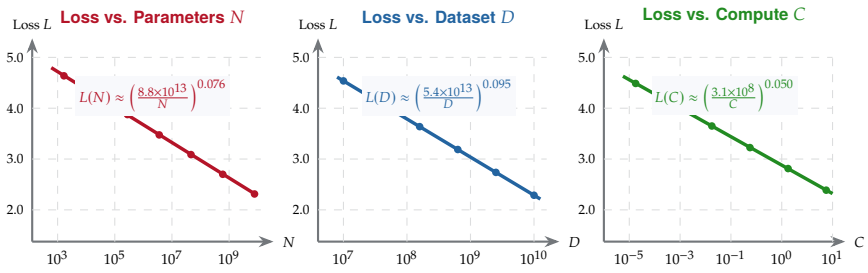



Figure 1.3: The three empirical scaling power laws from Kaplan et al. [**?** ]. Cross-entropy test loss $L$ scales smoothly as a power law with model parameter count $N$, dataset size $D$, and training compute $C$ across more than six orders of magnitude when performance is unconstrained by the other factors.

### 1.2.3 The breakthrough of empirical scaling laws (Kaplan et al., 2020)

The continuous, monotonic improvements observed across the GPT-2 model hierarchy prompted researchers at OpenAI to investigate whether neural language model performance was governed by quantitative physical regularities. In January 2020, Kaplan et al. [**?** ] published *"Scaling Laws for Neural Language Models"*, establishing that language modeling loss adheres to precise, predictable empirical power laws spanning many orders of magnitude.

Kaplan et al. conducted thousands of training runs spanning over six orders of magnitude in compute budget ($C \in [10^{17}, 10^{23}]$ FLOPs, or $10^{-5}$ to $10^{0}$ PF-days), model parameter capacity ($N \in [10^{3}, 10^{9}]$ non-embedding parameters), and training dataset volume ($D \in [10^{7}, 10^{10}]$ tokens). Their investigations revealed that cross-entropy test loss $L$ scales as a power law with each scale factor when performance is unconstrained by the other two:

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad \text{for } D \gg D_{\text{opt}}(N), \tag{1.11}$$

$$L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad \text{for } N \gg N_{\text{opt}}(D), \tag{1.12}$$

$$L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}, \quad \text{for optimal allocation of } (N, D), \tag{1.13}$$

where $N_c \approx 8.8 \times 10^{13}$ parameters, $D_c \approx 5.4 \times 10^{13}$ tokens, and $C_c \approx 3.1 \times 10^{8}$ PF-days denote constant scale factors, with fitted power-law exponents:

$$\alpha_N \approx 0.076, \quad \alpha_D \approx 0.095, \quad \alpha_C \approx 0.050. \tag{1.14}$$

Figure 1.3 illustrates these empirical relationships. On log-log coordinates, the test loss forms straight, parallel trajectories across six orders of magnitude. The authors noted that performance exhibits remarkable *hyperparameter insensitivity*: within broad boundaries, network shape hyperparameters (such as depth $L$ versus width $d_{\text{model}}$, feed-forward expansion ratios, or number of attention heads) exert negligible influence on test loss when total non-embedding parameter count $N$ is held fixed.

Beyond basic power laws, Kaplan et al. investigated *sample efficiency* and the dynamics of compute-efficient training. As illustrated in Panel A of Figure 1.4, their findings demonstrated that larger models are substantially more sample-efficient than smaller models: a large model reaches any target test loss with orders of magnitude fewer tokens processed and vastly fewer optimization steps. Consequently, **compute-efficient training stops far short of convergence**. Training a smaller model to convergence is computationally wasteful; it is far more compute-efficient to train a much larger model for fewer optimization steps, early-stopping before convergence.

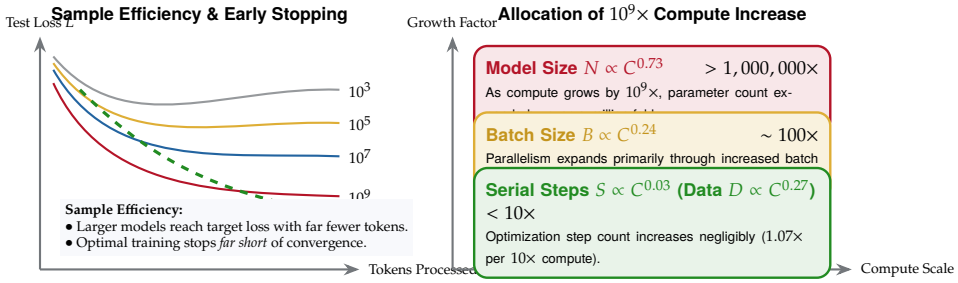



Figure 1.4: Sample efficiency and compute budget allocation in Kaplan et al. [**?** ]. Left: Larger models require orders of magnitude fewer tokens to attain an equivalent test loss, proving that compute-efficient training stops far short of full convergence. Right: Over a billion-fold ($10^9\times$) expansion in compute budget, Kaplan's empirical allocation dictates that over $1,000,000\times$ of the budget be spent on expanding model capacity $N$, while serial training steps increase by less than $10\times$.

From these dynamics, Kaplan et al. derived the empirically optimal allocation of compute. Modeling total floating-point operations as $C \approx 6ND = 6NBS$ (where $B$ is batch size and $S$ is the number of parameter update steps), the authors analyzed how an expanding compute budget should be partitioned between model size $N$, batch size $B$, and serial training steps $S$. Under optimal allocation at the critical batch size $B_{\text{crit}}$, they derived:

$$N \propto C^{0.73}, \quad B \propto C^{0.24}, \quad S \propto C^{0.03}, \quad \text{yielding} \quad D = B \cdot S \propto C^{0.27}. \tag{1.15}$$

Panel B of Figure 1.4 illustrates the physical consequence of Equation (1.15) over a billion-fold ($10^9\times$) increase in computational budget. To make optimal use of compute:

- Model parameter capacity $N$ should increase by more than **one million-fold** ($> 1,000,000\times$).
- Batch size $B$ should expand by approximately **one hundred-fold** ($\sim 100\times$).
- Serial optimization steps $S$ should increase by **less than ten-fold** ($< 10\times$).

In practical engineering terms, Kaplan et al. concluded: *"As the computational budget $C$ increases, it should be spent primarily on larger models, without dramatic increases in training time or dataset size."* Specifically, a $10\times$ increase in compute budget prescribed scaling model size $N$ by $5.4\times$, while increasing training tokens $D$ by merely $1.8\times$ and optimization steps by $1.07\times$. This theoretical prescription directly catalyzed the industry race toward colossal parameter regimes trained on relatively modest token horizons.

## 1.3 GPT-3: In-Context Learning Dynamics and Parameter Scaling

In June 2020, Brown et al. [**?** ] from OpenAI published *"Language Models are Few-Shot Learners"*, unveiling GPT-3. Building directly upon the scaling laws of Kaplan et al., GPT-3 scaled model capacity to 175 billion parameters—an order of magnitude larger than any previously trained dense autoregressive model.

### 1.3.1 Model topology, 175B capacity, and filtered corpus mixture

The GPT-3 architecture preserved the core decoder-only Transformer design of GPT-2, incorporating alternating dense and locally banded sparse self-attention sub-layers:

- **Depth and Width**: $L = 96$ Transformer layers, each containing $H = 96$ attention heads with per-head projection dimension $d_{\text{head}} = 128$.

- **Hidden Dimensionality**: Hidden representation dimension $d_{\text{model}} = 12,288$, with position-wise feed-forward intermediate dimensionality $d_{\text{ff}} = 4 \times 12,288 = 49,152$.
- **Context Window and Parameters**: Context window expanded to $L = 2048$ tokens, with a total model capacity of $N = 175$B parameters.

To train a 175B parameter network, OpenAI assembled a multi-source training mixture totaling 300 billion tokens. Given that raw web data contains significant noise, spam, and low-quality text, the authors implemented an aggressive quality filtering and differential sampling strategy:

1. **Filtered Common Crawl** (45 TB raw $\rightarrow$ 570 GB cleaned; 410B raw tokens): Filtered using a binary fastText classifier trained to discriminate curated WebText from raw web pages. During pre-training, Common Crawl was sampled at a rate of 60% (accounting for 180B tokens).
2. **WebText2** (19B tokens): An expanded version of WebText, sampled at 22% (66B tokens).
3. **Books1 and Books2** (12B and 55B tokens): Curated literary corpora, each sampled at 8% (24B tokens each).
4. **Wikipedia** (3B tokens): High-quality encyclopedic text, sampled at 3% (9B tokens).

High-quality, curated datasets were intentionally oversampled across multiple epochs, while lower-quality web scrapes were downsampled, establishing dataset composition as a first-order hyperparameter in large-scale pre-training.

### 1.3.2 In-context learning mechanics across the prompting spectrum

The central conceptual breakthrough of the GPT-3 paper was the formalization of *in-context learning* (ICL) as an operational alternative to supervised fine-tuning.

In the classical fine-tuning paradigm, model parameters $\boldsymbol{\Theta}$ are modified using gradient descent on task-specific labeled datasets:

$$\boldsymbol{\Theta}_{t+1} = \boldsymbol{\Theta}_t - \eta \nabla_{\boldsymbol{\Theta}} \mathcal{L}_{\text{task}}(\boldsymbol{\Theta}_t; \mathcal{D}_{\text{train}}). \tag{1.16}$$

While effective, fine-tuning introduces severe practical limitations: it requires curated labeled datasets for every task, risks catastrophic forgetting of general reasoning capabilities, and demands storing separate fine-tuned parameter weights for each application.

In contrast, under in-context learning, the pre-trained model parameters $\boldsymbol{\Theta}$ remain strictly frozen:

$$\nabla_{\boldsymbol{\Theta}} \mathcal{L} = \mathbf{0}. \tag{1.17}$$

The model executes task adaptation entirely in inference mode via feed-forward attention passes. Conditioning instructions and demonstration exemplars are provided directly inside the input prompt, and the model predicts the task completion through autoregressive decoding:

$$\hat{y} = \arg\max_{y \in \mathcal{V}^*} P(y \mid \text{Prompt}; \boldsymbol{\Theta}), \tag{1.18}$$

where Prompt = (Task Instruction, $(x_1, y_1), \ldots, (x_k, y_k), x_{\text{test}}$).

As illustrated in Figure 1.5, in-context learning spans three distinct operational regimes defined by the number of demonstration exemplars $k$ provided in the context window:

1. **Zero-Shot Learning ($k = 0$)**: The model receives only a natural language description of the task followed by the test prompt $x_{\text{test}}$. No input-output examples are supplied. This evaluates the model's ability to interpret abstract instructions zero-shot.

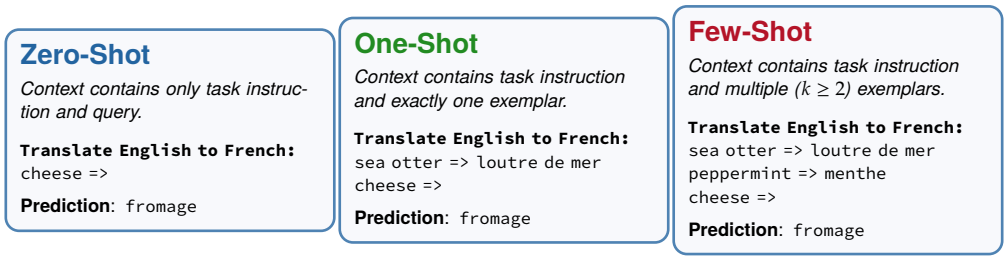



Figure 1.5: The prompting spectrum formalized in GPT-3 [**?** ]. The model solves tasks without gradient updates by conditioning on zero, one, or multiple input-output demonstrations.

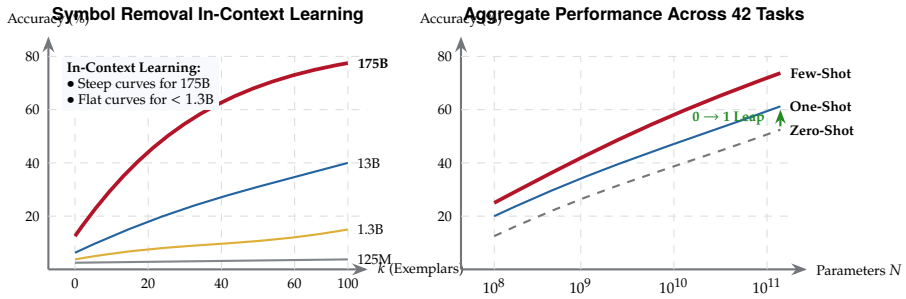



Figure 1.6: Empirical in-context performance scaling in GPT-3 [**?** ]. Left: In-context learning curves on the symbol removal task as a function of demonstration exemplars $k$, showing that the ability to rapidly extract task instructions emerges only at massive parameter scale (13B and 175B). Right: Aggregate accuracy across 42 evaluation benchmarks as a function of parameter scale, highlighting the prominent zero-to-one shot discontinuity and the widening advantage of few-shot conditioning.

2. **One-Shot Learning ($k = 1$)**: The model receives the task description along with exactly one demonstration exemplar $(x_1, y_1)$ prior to the test query $x_{\text{test}}$. The single exemplar serves as an explicit structural anchor, clarifying the expected formatting, delimiters, and target style.

3. **Few-Shot Learning ($k \geq 2$)**: The model receives the task description along with $k$ demonstration pairs:
$$((x_1, y_1), (x_2, y_2), \ldots, (x_k, y_k)),$$
where $k$ typically ranges from 10 to 100 depending on sequence length limits.

Figure 1.6 highlights the empirical dynamics of in-context learning across the GPT-3 model hierarchy (ranging from 125M, 350M, 760M, 1.3B, 2.7B, 6.7B, 13B, up to 175B parameters):

- **In-Context Learning as an Emergent Capability of Scale**: Smaller model variants (125M to 1.3B) exhibit near-flat in-context learning curves. As demonstrated on synthetic tasks such as symbol removal (Panel A of Figure 1.6), where the network must strip extraneous punctuation characters from corrupted words, small models fail to adapt regardless of how many exemplars $k$ are supplied. However, as parameter capacity scales to 13B and 175B, the slope of the learning curve steepens dramatically, climbing from $\sim 15\%$ zero-shot accuracy to $> 80\%$ accuracy with $k = 100$ context demonstrations.

- **The Zero-to-One Shot Discontinuity**: Across nearly all evaluated natural language tasks (Panel B of Figure 1.6), the single steepest marginal gain in accuracy occurs when moving from zero-shot to one-shot ($k = 0 \rightarrow k = 1$). Providing a single concrete demonstration resolves ambiguity regarding output formatting, syntax, and task delimiters, yielding a massive performance leap across all model scales.

- **Widening Gap with Demonstrations**: While zero-shot performance improves steadily with parameter capacity, few-shot performance scales significantly faster, creating an expanding performance gap that favors massive models conditioned on multiple context exemplars.

### 1.3.3 Empirical benchmark performance and comparison with fine-tuned models

The most significant empirical finding of Brown et al. [**?** ] was that a frozen 175B parameter language model conditioned purely via few-shot demonstrations could achieve performance competitive with, and in several cases superior to, task-specific state-of-the-art models fine-tuned on hundreds of thousands of labeled examples.

Table 1.2 provides an extensive quantitative comparison across closed-book question answering, machine translation, commonsense reasoning, and reading comprehension benchmarks.

A rigorous examination of Table 1.2 provides vital insights into the strengths and boundaries of in-context scaling:

- **Superhuman Closed-Book Retrieval on TriviaQA**: In the closed-book setting, models must answer open-domain factual queries without access to external documents or retrieval search engines. On TriviaQA, GPT-3 175B zero-shot (64.3%) already outperformed the fine-tuned closed-book T5-11B baseline (50.1%), while few-shot GPT-3 attained 71.2%, surpassing even the state-of-the-art open-domain retrieval-augmented RAG model (68.0%). This proved that scaling parameter capacity to 175B allows dense feed-forward networks to act as implicit factual knowledge bases, storing and retrieving world facts directly within weight matrices.

- **Unsupervised Machine Translation into English**: Without any parallel training corpora, GPT-3 few-shot achieved 39.2 BLEU translating French into English (surpassing supervised SOTA by +4.2 BLEU) and 40.6 BLEU translating German into English (matching/exceeding supervised SOTA). On Romanian-to-English, GPT-3 reached 39.5 BLEU, within 0.4 BLEU of a supervised system trained on 608,000 labeled sentence pairs and back-translation. However, translating *out* of English into foreign languages exhibited noticeably lower BLEU scores, reflecting the heavy English skew (> 90%) of the pre-training corpus.

- **Commonsense Physical Reasoning on PIQA**: On the Physical Interaction QA (PIQA) dataset, GPT-3 scored 82.8% in the few-shot setting, setting a new overall state-of-the-art that exceeded the prior fine-tuned RoBERTa model (79.4%).

- **The Failure Modes: Discrete Reasoning and Span Extraction**: While GPT-3 dominated factual retrieval and text synthesis, it lagged behind specialized fine-tuned models on DROP (36.5 F1 vs. SOTA 89.1) and SQuAD v2.0 (69.8 F1 vs. SOTA 93.0). Tasks on DROP require multi-step discrete numerical operations (counting, sorting, addition) over paragraph spans, which standard autoregressive token generation cannot reliably solve without explicit scratchpads or chain-of-thought prompting. Similarly, extractive span QA models trained specifically to predict start and end boundary logits naturally outperform generative token decoding on exact span matching.

## 1.4 The Race to Bigger Models and the Chinchilla Scaling Revolution

Following the publication of GPT-3 and the confirmation that model loss scales smoothly with parameter capacity, the natural language processing industry embarked on an aggressive race to construct ever-larger dense neural networks. Between 2020 and 2022, leading research institutions treated parameter count as the primary determinant of model intelligence.

### 1.4.1 The race to massive scale and the undertraining diagnosis

During this era, major industrial laboratories deployed dense autoregressive models well exceeding 100 billion parameters:

- **Jurassic-1** (AI21 Labs, 2021): 178 billion parameters, evaluated on broad linguistic benchmarks.
- **Gopher** (DeepMind, 2021): 280 billion parameters, trained on 300 billion tokens with a compute budget of $C = 5.76 \times 10^{23}$ FLOPs [**?** ].
- **Megatron-Turing NLG** (NVIDIA and Microsoft, 2021): 530 billion parameters, trained across massive distributed GPU clusters using 3D parallelism [**?** ].
- **PaLM** (Google, 2022): 540 billion parameters, trained using the Pathways distributed compiler across dual TPU v4 pods on 780 billion tokens [**?** ].

During this period, foundation model papers adopted a distinctive structure: extensive catalogs of empirical benchmark results (frequently spanning over 100 pages) accompanied by brief architectural descriptions. However, operational severe bottlenecks emerged: deploying a 500B+ parameter model required multi-node tensor parallelism, massive GPU cluster interconnects, and prohibitive inference memory bandwidth, making real-world serving economically unsustainable.

In March 2022, Hoffmann et al. [**?** ] from DeepMind published *"Training Compute-Optimal Large Language Models"*, introducing the **Chinchilla** architecture. Their central thesis challenged the industry consensus:

> *"Current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping the training data constant."*

Hoffmann et al. diagnosed a fundamental methodological flaw in the empirical framework of Kaplan et al. [**?** ]. Specifically, Kaplan's experiments trained models using a fixed cosine learning rate schedule that decayed learning rates across a constant step horizon. Consequently, models trained for longer durations were evaluated using suboptimal learning rates, which artificially depressed their performance and made them appear less compute-efficient. Furthermore, Kaplan kept training dataset sizes relatively constant ($D \approx 300$B tokens) while expanding parameter counts ($N \rightarrow 175$B), creating a massive data starvation bottleneck.

Re-evaluating the compute-optimal frontier, DeepMind proved that parameter count $N$ and training dataset size $D$ should be scaled in *equal proportion*:

$$N \propto C^{0.5}, \quad D \propto C^{0.5}. \tag{1.19}$$

That is: **for every doubling of model parameters, the number of training tokens must also be doubled.**

### 1.4.2 The three methodological approaches to compute-optimal scaling

To establish the compute-optimal scaling frontier, Hoffmann et al. trained over 400 models spanning parameter counts from 70M to 16B and dataset sizes from 5B to 500B tokens across three distinct methodological approaches.

**Approach 1: Minimum Loss Envelopes over Training Curves** In Approach 1, the authors varied training duration and compute budget for models of fixed parameter capacity (70M to 10B parameters), matching the cosine learning rate schedule length to each specific run. By plotting training curves across FLOP milestones and extracting the lower boundary envelope

across all curves, they identified the model size that achieved minimal loss for each compute budget. Fitting power-law equations $N_{\text{opt}} \propto C^{a}$ and $D_{\text{opt}} \propto C^{b}$ on log-log coordinates yielded:

$$N_{\text{opt}} \propto C^{0.49}, \quad D_{\text{opt}} \propto C^{0.51}. \tag{1.20}$$

When applied to Gopher's computational budget ($C = 5.76 \times 10^{23}$ FLOPs), Approach 1 demonstrates that the optimal model size is **67 billion parameters**, rather than the 280 billion instantiated in Gopher.

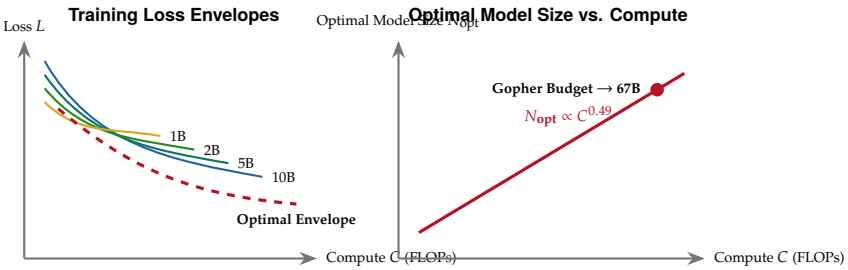



Figure 1.7: Chinchilla Approach 1: Minimum loss envelopes over training curves [**?** ]. By extracting the optimal parameter size at the lower boundary envelope for each compute budget, a linear fit on log-log coordinates establishes $N_{\text{opt}} \propto C^{0.49}$, indicating that Gopher's compute budget corresponds to an optimal model size of 67B parameters.

**Approach 2: IsoFLOP Profiles** In Approach 2, compute budgets were held strictly constant across discrete slices ($C \in [6 \times 10^{18}, 3 \times 10^{21}]$ FLOPs). For each slice, parameter count $N$ was varied while setting token count according to $D = C/(6N)$. Plotting final validation loss against parameter size revealed a convex parabolic profile: undersized models suffer from insufficient capacity, while oversized models suffer from undertraining. The vertex of each parabola defines the optimal parameter size. Fitting a power-law to these minima yielded:

$$N_{\text{opt}} \propto C^{0.49}, \quad D_{\text{opt}} \propto C^{0.51}. \tag{1.21}$$

Evaluating this fit at Gopher's compute allocation yields an optimal model size of **63 billion parameters**.

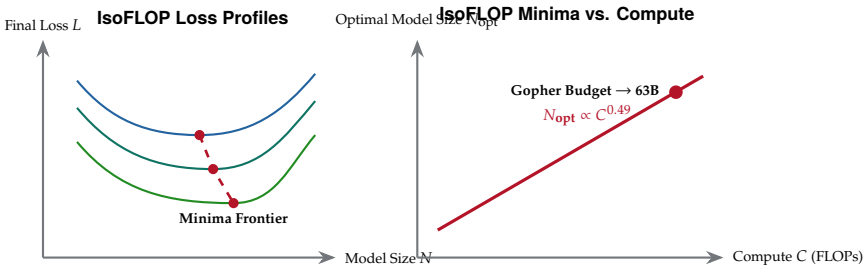



Figure 1.8: Chinchilla Approach 2: IsoFLOP profiles [**?** ]. For a constant compute budget, plotting final loss against parameter count yields a convex parabolic profile. The minimum of each parabola identifies the optimal model size, yielding $N_{\text{opt}} \propto C^{0.49}$ and indicating an optimal Gopher scale of 63B.

**Approach 3: Parametric Loss Surface Modeling** In Approach 3, Hoffmann et al. formalized the continuous loss surface via a joint parametric formulation:

$$\hat{L}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \tag{1.22}$$

where $E = 1.69$ is the irreducible natural entropy of text, $A/N^{\alpha}$ is the penalty for finite parameter capacity, and $B/D^{\beta}$ is the penalty for finite training data. Fitting via Huber loss minimization yielded:

$$A = 406.4, \quad B = 410.7, \quad \alpha = 0.34, \quad \beta = 0.28. \tag{1.23}$$

Solving the constrained optimization $\min_{N,D} \hat{L}(N, D)$ subject to $6ND = C$ via Lagrange multipliers yields:

$$N_{\text{opt}} \propto C^{\frac{\beta}{\alpha+\beta}} = C^{\frac{0.28}{0.34+0.28}} = C^{0.45}, \quad D_{\text{opt}} \propto C^{\frac{\alpha}{\alpha+\beta}} = C^{\frac{0.34}{0.34+0.28}} = C^{0.55}. \tag{1.24}$$

Evaluating Approach 3 for Gopher's compute allocation yields an optimal model size between **40 and 70 billion parameters**.

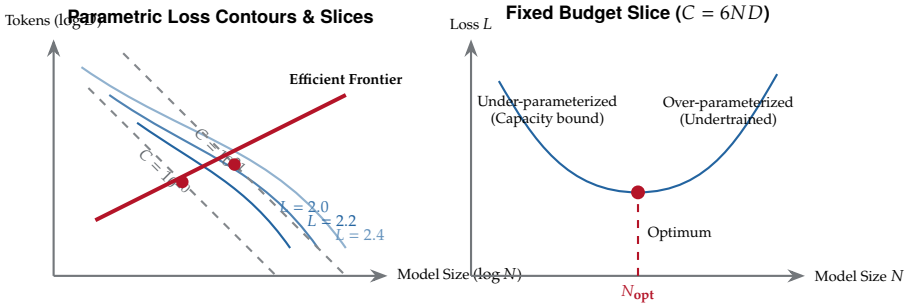



Figure 1.9: Chinchilla Approach 3: Parametric loss surface modeling [**?** ]. Left: Continuous loss contours generated by fitting Equation (1.22). For each fixed compute budget line $C = 6ND$, optimal allocation occurs where the budget line is tangent to a loss contour, tracing the Efficient Frontier ($N_{\text{opt}} \propto C^{0.45}$). Right: Vertical slice at fixed compute budget demonstrating that deviation from optimal parameter capacity yields elevated loss due to either under-parameterization or undertraining.

### 1.4.3 Empirical validation: Chinchilla (70B) versus Gopher (280B)

To empirically test their derivations, DeepMind trained **Chinchilla**, a 70-billion-parameter model using the exact same compute budget as Gopher ($C = 5.76 \times 10^{23}$ FLOPs). Table 1.3 highlights the structural resource allocation between Chinchilla and contemporary oversized architectures.

While Gopher allocated its budget to maintain 280 billion parameters trained on 300 billion tokens, Chinchilla used a model 1/4× the size (70B) trained on over 4× more data (1.4 trillion tokens).

Despite having only one-fourth of Gopher's parameter count, Chinchilla uniformly outperformed Gopher, GPT-3, Jurassic-1, and Megatron-Turing NLG across an extensive suite of downstream benchmarks:

- **MMLU (Massive Multitask Language Understanding)**: Chinchilla achieved 67.5% average accuracy, outperforming Gopher (60.0%) and GPT-3 (53.9%).
- **BIG-bench**: Chinchilla achieved a 10.7% absolute performance improvement over Gopher.
- **Commonsense and Reading Comprehension**: Chinchilla outperformed GPT-3 and Gopher on LAMBADA, PIQA, and BoolQ.

The Chinchilla experiments conclusively refuted the Kaplan allocation rule, proving that the post-GPT-3 race toward massive parameter sizes had produced oversized, undertrained architectures.

## 1.5 The Modern Paradigm Shift: Token Saturation and Open Foundation Models

The Chinchilla breakthrough prompted an industry-wide transition in language model training. Rather than continuing parameter escalation (500B $\rightarrow$ 1T), the field pivoted toward training compact models on massive token volumes. This shift made high-performance language models vastly more accessible, catalyzing the modern open-weight ecosystem.

### 1.5.1 Token over-training and inference economics

While Chinchilla established the *compute-optimal training* ratio ($N \propto D$), real-world deployment depends heavily on *inference efficiency*.

In a production environment, training compute is amortized over billions of downstream user queries. A 70B parameter model requires substantially more GPU memory bandwidth, higher latency, and complex multi-GPU tensor parallelism during inference compared to an 8B parameter model. Consequently, training a smaller model (e.g., 7B or 8B) far beyond the Chinchilla compute-optimal frontier—training on 2 trillion to 15 trillion tokens (*token over-training*)—incurs a higher initial training cost, but yields dramatically lower per-token latency and memory requirements at inference time.

Token over-training allowed high-performance foundation models to fit within the memory capacity of single workstation GPUs, consumer graphics cards, and edge hardware, laying the groundwork for widespread open-weight adoption.

### 1.5.2 The LLaMA ecosystem and generational evolution

In February 2023, Touvron et al. [**?** ] from Meta AI introduced **LLaMA** (*Large Language Model Meta AI*), demonstrating that open foundation models trained exclusively on publicly available datasets could match or exceed proprietary models.

The LLaMA series evolved through successive generations:

1. **LLaMA-1 (February 2023)**: Released in four sizes (7B, 13B, 33B, 65B) trained on 1.0T to 1.4T tokens. The 13B model outperformed GPT-3 (175B) across most benchmarks while fitting on a single consumer GPU, while LLaMA-65B rivaled Chinchilla-70B and PaLM-540B.

2. **Llama 2 (July 2023)**: Trained on 2.0T tokens with an expanded context window of 4096 tokens. The architecture introduced Grouped-Query Attention (GQA) for the 70B model to reduce key-value cache memory overhead during autoregressive generation [**?** ]. Meta released specialized fine-tuned checkpoints, including Llama-2-Chat (aligned via Reinforcement Learning from Human Feedback) and Code Llama (specialized for software engineering).

3. **Llama 3 and 3.1 (April/July 2024)**: Scaled training data to over 15 trillion tokens with an expanded 128k context window and a 128k tiktoken vocabulary. The release featured an 8B model, a 70B model, and the flagship **405B parameter frontier model**, trained across a cluster of 16,000 H100 GPUs using 4D parallelism (tensor, pipeline, context, and data parallelism).

4. **Llama 3.2 (September 2024)**: Introduced lightweight edge models (1B and 3B parameters) optimized for on-device inference, alongside multimodal vision-language models (11B and 90B parameters) capable of processing image-text inputs.

Meta released the LLaMA weights under a permissive community license, with an explicit commercial clause requiring custom licensing only for platforms exceeding 100 million daily

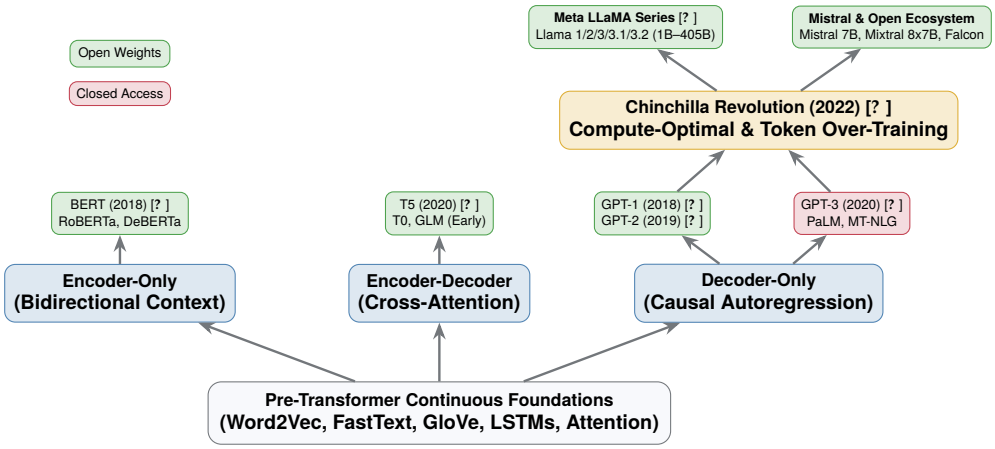



Figure 1.10: The macro-evolutionary tree of language models. Following the introduction of the Transformer in 2017, the field branched into encoder-only, encoder-decoder, and decoder-only architectures. The decoder-only branch expanded rapidly from GPT-1 and GPT-2 to massive proprietary models, before the Chinchilla compute-optimal revolution redirected development toward token-saturated open foundation models.

active users (effectively restricting competing hyperscalers while providing broad access to academic and commercial developers).

### 1.5.3 Open foundation model families and the macro-evolutionary tree

The success of the LLaMA family catalyzed a broader ecosystem of open foundation models:

- **EleutherAI (GPT-Neo, GPT-J, GPT-NeoX)**: An open-source research collective that championed accessible foundation models by reverse-engineering early GPT architectures and releasing **The Pile** (an 825 GB diverse text corpus).
- **Mistral AI (France)**: Founded in 2023, Mistral introduced compact, high-efficiency models utilizing Sliding Window Attention. Key releases include **Mistral 7B** (outperforming Llama 2 13B), **Mixtral 8x7B** (a sparse Mixture-of-Experts model that routes tokens dynamically through 8 feed-forward experts, activating only 2 experts per token to deliver 47B capacity with 13B active compute speed), and **Pixtral** (multimodal vision capabilities).
- **GLM (Zhipu AI and Tsinghua University, China)**: The General Language Model family, designed around an autoregressive blank-filling objective to unify bidirectional and causal modeling. GLM models emphasize bilingual proficiency across Chinese and English.
- **Falcon (Technology Innovation Institute, UAE)**: Falcon 7B, 40B, and 180B, trained on the custom-curated RefinedWeb corpus. The Falcon initiative also pioneered evaluations of non-Transformer sequence models, including state-space architectures (Mamba).

Figure 1.10 synthesizes the architectural divergence of language models from pre-Transformer continuous representations into the three modern branches, illustrating the evolution of open-weight and closed-access systems.

As shown in Figure 1.10, language modeling development converged decisively toward the decoder-only paradigm. While encoder-only models stabilized after dominating classification and embedding benchmarks, decoder-only models became the universal architecture for multi-task reasoning, zero-shot transfer, and autonomous agents. The Chinchilla compute-optimal revolution redirected this trajectory from pure parameter scaling to token saturation, democratizing foundation model deployment across the global open-source community.

Table 1.2: Empirical evaluation of GPT-3 (175B) across Zero-Shot, One-Shot, and Few-Shot settings compared to prior fine-tuned task-specific state-of-the-art models [**?** ].

| Benchmark / Task | Metric | Fine-Tuned SOTA | GPT-3 (0-Shot) | GPT-3 (1-Shot) | GPT-3 (Few-Shot) |
|---|---|---|---|---|---|
| **Closed-Book Question Answering** | | | | | |
| **TriviaQA** | Acc (%) | 68.0 / 60.5* | 64.3 | 68.0 | **71.2** |
| **WebQuestions** | Acc (%) | 45.5 / 44.7* | 14.4 | 25.3 | 41.5 |
| **Natural Questions** | Acc (%) | 44.5 / 36.6* | 14.6 | 23.0 | 29.9 |
| **Machine Translation (WMT)** | | | | | |
| **WMT'14 Fr → En** | BLEU | 35.0 | 21.2 | 33.7 | **39.2** |
| **WMT'16 De → En** | BLEU | 40.2 | 27.2 | 30.4 | **40.6** |
| **WMT'16 Ro → En** | BLEU | 39.9 | 19.9 | 38.6 | 39.5 |
| **Commonsense Reasoning & Winograd** | | | | | |
| **PIQA** | Acc (%) | 79.4 | 80.5 | 80.5 | **82.8** |
| **WSC273 (Winograd)** | Acc (%) | 90.1 | 88.3 | 89.7 | 88.6 |
| **Winogrande (Adversarial)** | Acc (%) | 84.6 / 79.0† | 70.2 | 73.2 | 77.7 |
| **ARC (Easy)** | Acc (%) | 92.0 | 68.8 | 71.2 | 70.1 |
| **ARC (Challenge)** | Acc (%) | 78.5 | 51.4 | 53.2 | 51.5 |

Table 1.3: Comparison of compute-optimal Chinchilla against oversized architectures under equivalent or comparable training compute budgets [**?** ].

| **Model** | **Parameters ($N$)** | **Tokens ($D$)** | **FLOPs ($C$)** | **Relative Sizing Regime** |
|---|---|---|---|---|
| **GPT-3** | 175B | 300B | $3.15 \times 10^{23}$ | Severely undertrained ($N \gg D_{\text{opt}}$) |
| **Jurassic-1** | 178B | 300B | $3.20 \times 10^{23}$ | Severely undertrained ($N \gg D_{\text{opt}}$) |
| **Gopher** | 280B | 300B | $5.76 \times 10^{23}$ | Severely undertrained (4× optimal size) |
| **Megatron-Turing** | 530B | 270B | $8.59 \times 10^{23}$ | Extreme parameter over-allocation |
| **Chinchilla** | **70B** | **1400B (1.4T)** | $5.76 \times 10^{23}$ | **Compute-Optimal Frontier** |